IMPORTANT - NOTES

The Self-Assignment Problem

Self assignment suppose:

```
Stud s1(90);
we do -> s1 = s1;
```

both source and destination are same, for simple integer harmless, but what if:

```
Stud & operator=(const Stud &obj) {
    delete marks;
    marks = new int(*obj.marks);
    return *this;
}
```

> **Undefined Behaviour Alert:**
> After deletion `s1.marks` is a dangling pointer, trying to read freed memory -> this is undefined behaviour.

thats why its a good practice to check for self assignment

```
Stud & operator=( ) {
    if (this == &obj) { return *this; }
    delete marks;
    marks = new int(*obj.marks);
    return *this;
}
```

Chained Assignments

in Copy Assignment operator return type is `Student &` and the function returns `*this`. so overall we return [destination object itself by reference].

reason `s4 = s3 = s2 = s1`
C++ evaluates R to L `s4 = s3 = (s2 = s1)` and `s2` is returned by reference. this allows `(s3 = s2)` instead of halting the chain at `s4 = s3 =` (pass by value) takes place.

Deep Copy Implementation

Implementing a Deep copy constructor

```
Stud(const Stud &obj) {
    marks = new int(*obj.marks);
}
```

Implementing a Deep copy assignment constructor

```
Stud & operator=(const Stud &obj) {
    if (this == &obj) { return *this; }
    delete marks;
    marks = new int(*obj.marks);
    return *this;
}
```

The Rule of 3

Rule of 3:

If a class requires a user defined destructor, copy constructor, or copy assignment operator, it will require all 3.

because if we implement:

- **only destructor**

  -> default copying can cause shared ownership / double delete
- **only copy constructor**

  -> default assignment can still cause shallow copying

- **only copy assignment**

  -> default copy constr can still cause shallow copying

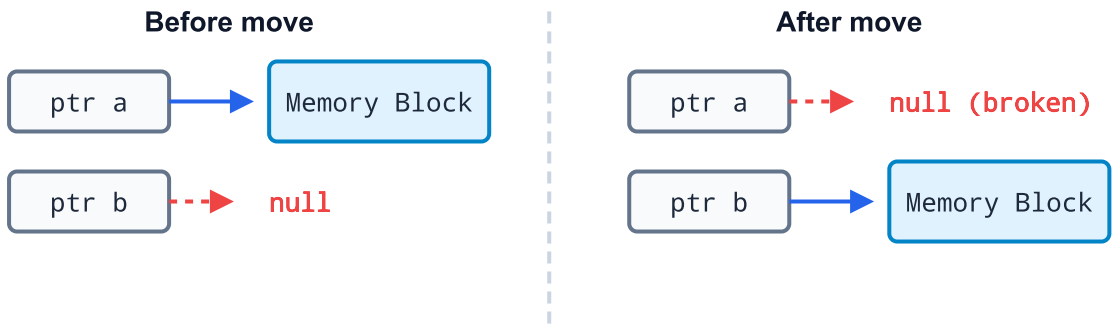
Move Semantics (Move Constructor & Assignment)

Problem with copying

```
vector<int> v1(10000, 10);
vector<int> v2 = v1;
```

`v1` owns a dynamically allocated array, `v2` gets its own separate array, addition time and memory.

Move



Note: Move is a method where resource ownership is transferred instead of duplicating resources. It avoids allocating new memory.

Move Constructor

creates a new object by taking resources from existing object
Syntax: `ClassName (ClassName && obj)` (`&&` -> rvalue reference)

Eg

```
Stud (Stud && obj) {
    marks = obj.marks; // -> transfers the pointer address to the destination
    obj.marks = nullptr; // -> remove source object ownership of that resource
(it's not destroyed, just ownership is transferred)
}

now
Stud s2 (move(s1)) // in main
```

> **Tip:** `move()` does not itself move the data. It allows the object to be treated as an rvalue, move transfer happens inside constructor.

Move Assignment

```
Stud & operator=(Stud && obj) {
    if (this == &obj) { return *this; }
    delete marks;
    marks = obj.marks;
    obj.marks = nullptr;
    return *this;
}

and in main
Stud s1(90);
Stud s2(5);
s2 = move(s1);
```

what it does is:

1. deletes memory previously owned by `s2`
2. transfer pointer from `s1` to `s2`
3. set source pointer to `nullptr` so that source destructor cant delete transferred resource
4. return destination object by reference

Rule of 5 & Rule of 0

If a class needs one of these special member function, it needs all 5 of them.

Special Member Function	Purpose / Action
destructor	release resource
copy constr	deep copy into new object
copy assign	deep copy into existing object
move constructor	transfer resource into new object
move assign	transfer resource into existing object

Rule of 0:

instead of manually managing memory using raw pointers, destructor, copy constructor, copy assign, move constructor, move assign, prefer resource managing types such as `vector`, `strings` etc which have their own resource management. no need to write 5 special functions ourself. this is rule of 0.